

Hyperparamètres en machine learning

Antisèche de formation — modèle, hyperparamètres clés, valeurs de départ et impact du réglage. Valeurs indicatives (conventions scikit-learn / XGBoost). Les flèches ↑ / ↓ signifient « augmenter / diminuer ce paramètre ».

Avant de régler quoi que ce soit

1. Établir une **baseline** avec les valeurs par défaut : c'est la référence à battre.
2. Pipeline propre d'abord : découpage train / validation / test, mise à l'échelle, validation croisée.
3. Régler **2 à 3 hyperparamètres à fort impact**, un changement à la fois, mesuré sur la validation.
4. Le jeu de **test** ne sert qu'à la toute fin, jamais pour choisir un modèle ou un réglage.

Sur quoi optimiser : les métriques de validation

Régler un modèle n'a de sens qu'au regard d'une métrique cible, choisie selon le problème.

Problème	Métrique principale
Régression	RMSE, MAE, R^2
Classification équilibrée	Accuracy
Classification déséquilibrée	F1, ROC-AUC, PR-AUC
Clustering	Silhouette score

Sur données déséquilibrées, l'accuracy trompe : un modèle qui prédit toujours la classe majoritaire peut afficher 95 % sans rien apprendre — préférer F1 ou PR-AUC.

Carte des familles de modèles

Apprentissage supervisé

on apprend à partir de données étiquetées ($X \rightarrow y$)

Linéaires

régl. linéaire, logistique
frontière linéaire

Voisinage

KNN
vote des k voisins proches

Marge

SVM
séparateur à marge maximale

Arbres

arbre de décision
règles si / alors

Ensembles

Random Forest, boosting
agrège plusieurs arbres

Réseaux

MLP, deep learning
couches de neurones

Apprentissage non supervisé

aucune étiquette, on cherche une structure dans les données

Clustering

K-Means
regroupe sans étiquette ·
segmentation

Cheat sheet par modèle

Régression linéaire · Ridge / Lasso

Linéaire

Hyperparamètre	Départ	Impact
alpha	1.0	Force de régularisation. ↑ = modèle plus simple, moins d'overfitting
l1_ratio (ElasticNet)	0.5	Dosage L1 / L2. Vers 1 = sélection de variables (sparsité)

Standardiser les features est indispensable dès qu'on régularise.

Régression logistique

Linéaire · classif.

Hyperparamètre	Départ	Impact
C	1.0	Inverse de la régularisation. ↓ C = plus de régularisation
penalty	l2	l1 pour de la sparsité, l2 par défaut
solver	lbfgs	saga sur gros volumes, liblinear pour petits jeux + l1
class_weight	None	'balanced' si classes déséquilibrées

k plus proches voisins (KNN)

Voisinage

Hyperparamètre	Départ	Impact
n_neighbors (k)	5	↑ k = frontière plus lisse (↑ biais) · ↓ k = sur-ajustement
weights	uniform	'distance' donne plus de poids aux voisins proches
metric	euclidienne	cosine pour du texte / des embeddings

Mise à l'échelle des features obligatoire (algorithme basé sur les distances).

SVM (machine à vecteurs de support)

Marge

Hyperparamètre	Départ	Impact
C	1.0	Tolérance aux erreurs. ↑ C = marge dure, risque d'overfit
kernel	rbf	linéaire si beaucoup de features, rbf sinon
gamma	'scale'	Portée d'un exemple. ↑ gamma = frontière plus sinueuse

Les SVM à noyau RBF deviennent vite coûteux sur de grands jeux de données ; envisager un modèle linéaire ou un ensemble d'arbres. Le seuil exact dépend de la dimension et du matériel.

Arbre de décision

Arbre

Hyperparamètre	Départ	Impact
max_depth	3-10	Profondeur. Principal levier anti-overfitting
min_samples_leaf	1	↑ (5-50) = arbre plus régularisé, plus lisse
min_samples_split	2	Seuil minimal pour scinder un nœud
max_features	None	Limiter pour décorrélérer les coupes

Random Forest

Ensemble · bagging

Hyperparamètre	Départ	Impact
n_estimators	100-300	↑ = plus stable (rendements décroissants, plus lent)
max_features	'sqrt'	Contrôle la décorrélation entre arbres
max_depth	None	Limiter si overfitting / données bruitées
min_samples_leaf	1	↑ pour lisser les prédictions
class_weight	None	'balanced' si classes déséquilibrées (levier souvent oublié)

Mettre `n_jobs=-1` pour paralléliser sur tous les cœurs.

Gradient Boosting · XGBoost / LightGBM

Ensemble · boosting

Hyperparamètre	Départ	Impact
learning_rate	0.05-0.1	↓ + ↑ n_estimators = meilleure généralisation
n_estimators	100-500	Couplé au learning_rate (compromis vitesse / perf)
max_depth	3-6	Arbres peu profonds (boosting = weak learners)
subsample	0.8	Échantillonnage des lignes, régularise
colsample_bytree	0.8	Échantillonnage des colonnes par arbre
min_child_weight	1	↑ = coupes plus prudentes (régularise)
gamma (min_split_loss)	0	Gain minimal pour couper. ↑ = arbres plus conservateurs
reg_alpha (L1)	0	Régularisation L1 sur les poids, induit de la sparsité
reg_lambda (L2)	1	Régularisation L2, pénalise les grands poids
scale_pos_weight	1	Classes déséquilibrées : régler au ratio négatifs / positifs

Utiliser `early_stopping` sur un set de validation pour fixer `n_estimators` automatiquement.

K-Means

Clustering

Hyperparamètre	Départ	Impact
n_clusters (k)	elbow / silhouette	Le choix structurant, à valider quantitativement
init	'k-means++'	Bien meilleure initialisation que 'random'
n_init	10 (ou 'auto')	Nombre de relances. ↑ = résultat plus robuste

Préférer la silhouette à l'elbow (souvent subjectif) quand les ressources le permettent. Standardiser les features ; sensible aux outliers et aux clusters non sphériques.

Réseau de neurones

Deep learning

Hyperparamètre	Départ	Impact
learning_rate	1e-3 (Adam)	LE paramètre clé. Trop haut = divergence, trop bas = lent
batch_size	32	↑ = plus stable / rapide · ↓ = plus bruité (peut régulariser)
epochs	+ early stopping	Arrêter quand la perte de validation remonte
optimizer	Adam	AdamW si weight decay · SGD + momentum en vision
dropout	0.2-0.5	Régularisation. ↑ si overfitting
couches / unités	petit puis élargir	↑ capacité si sous-apprentissage

AdamW est souvent le meilleur point de départ pour les problèmes tabulaires et le NLP modernes (Adam + weight decay correctement découplé).

Recherche d'hyperparamètres & ordre de réglage

Grid Search

Teste toutes les combinaisons d'une grille. Simple, mais explose vite — à réserver aux petits espaces.

Random Search

Tire des combinaisons au hasard. À budget égal, bat souvent la grid search exhaustive — à privilégier par défaut.

Bayésien · Optuna

Recherche guidée par les essais passés. Le plus efficace sur les grands espaces coûteux.

Toujours coupler à une validation croisée, et ne jamais optimiser sur le holdout.

Random Forest

1. n_estimators
2. max_depth
3. min_samples_leaf
4. max_features

XGBoost

1. learning_rate
2. n_estimators
3. max_depth
4. subsample · colsample_bytree
5. régularisation : min_child_weight, gamma, reg_alpha, reg_lambda

Le réglage n° 1. Si un seul hyperparamètre doit être réglé en premier sur un réseau de neurones ou un modèle de boosting : commencer par `learning_rate`. C'est le levier le plus déterminant de toute la fiche.

Glossaire des notions transversales

Paramètre

Appris automatiquement par le modèle pendant l'entraînement (poids, coefficients). On ne les fixe pas à la main.

Hyperparamètre

Fixé par nous avant l'entraînement (profondeur, taux d'apprentissage...). C'est ce qu'on règle.

régularisation

Pénalise la complexité du modèle pour limiter le sur-apprentissage. Se cache derrière α , C , λ , dropout, weight decay.

learning_rate · taux d'apprentissage

Taille des pas pendant l'optimisation. Trop grand = instable, trop petit = lent.

max_depth · profondeur

Jusqu'où un arbre se ramifie. Levier direct sur la capacité (et l'overfitting).

n_estimators

Nombre de modèles agrégés dans un ensemble : arbres d'une forêt, étapes de boosting.

batch_size

Nombre d'exemples vus avant chaque mise à jour des poids (réseaux de neurones).

epoch · époque

Un passage complet sur l'ensemble du jeu d'entraînement.

dropout

Désactive aléatoirement des neurones à l'entraînement pour régulariser le réseau.

kernel · noyau

Fonction qui projette les données pour les rendre séparables (SVM).

gamma

Portée d'influence d'un point de données dans un noyau RBF (SVM).

k

Selon le modèle : nombre de voisins (KNN) ou nombre de clusters (K-Means).

early stopping

Arrête l'entraînement dès que la performance de validation cesse de progresser.

validation croisée

Découpe le train en k plis pour estimer la performance de façon robuste, avant le test final.

Découpage des données

Train · 60 %

le modèle apprend ses paramètres dessus

Validation · 20 %

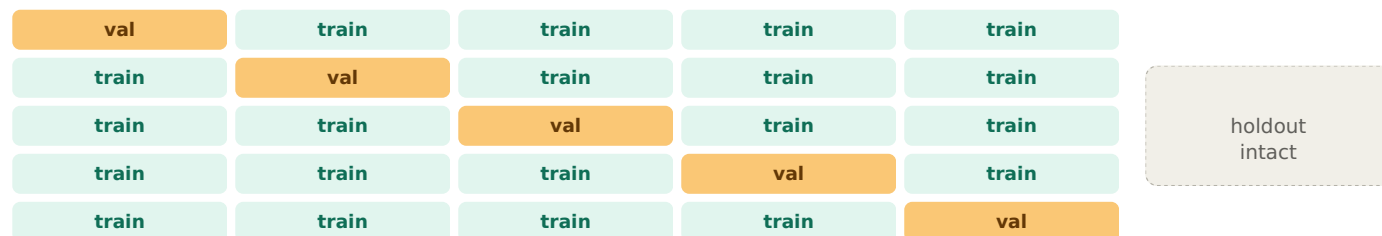
régler les hyperparamètres, comparer

Holdout · 20 %

scellé — ouvert une seule fois, à la fin

Exemple de ratio : 60/20/20 ou 70/15/15 selon la taille du jeu. Sur de gros volumes, on utilise souvent 80/20 (train/test) avec validation croisée à la place d'une validation fixe.

Validation croisée (k-fold) — uniquement sur train + validation ; le holdout reste à part.



■ train ■ validation ■ holdout scellé

Règle d'or. Si le holdout sert à choisir un modèle ou un hyperparamètre, il devient une 2^e validation : le score final est alors optimiste. La validation « à l'aveugle » n'a de valeur que parce qu'on ne l'a jamais regardée avant.

Diagnostic — quel hyperparamètre toucher ?

On diagnostique en comparant l'erreur sur le train et sur la validation.

Sous-apprentissage · biais élevé

Symptôme

Erreur élevée sur le train ET la validation, les deux proches l'une de l'autre.

Actions

Modèle plus complexe (↑ profondeur, ↑ couches / unités, ↑ `n_estimators`) · ↓ régularisation (↑ `C`, ↓ `alpha`, ↓ dropout) · enrichir les features · entraîner plus longtemps.

Sur-apprentissage · variance élevée

Symptôme

Erreur faible sur le train mais élevée sur la validation : grand écart entre les deux.

Actions

↑ régularisation (↓ `C`, ↑ `alpha`, ↑ dropout, weight decay) · plus de données / augmentation · early stopping · réduire la complexité (↓ profondeur, `max_features`) · simplifier.

Bon équilibre. Erreurs de train et de validation toutes deux basses et proches l'une de l'autre.